

CASE 03 · TEORIA DA COMPUTAÇÃO

PESQUISA TEÓRICA

ANALISADOR LÉXICO

Compilação · Análise Léxica · Tokens · Tabela de Símbolos

Disciplina: Teoria da Computação | 2025

TÓPICO 1.1

Compilação x
Interpretação

TÓPICO 1.2

Análise
Léxica

TÓPICO 1.3

Mecanismos
Envolvidos

TÓPICO 1.4

Tabela de
Símbolos

RESUMO

Esta pesquisa aborda os fundamentos dos analisadores léxicos, primeira fase de um compilador. São estudados os conceitos de compilação e interpretação, os mecanismos formais que sustentam a análise léxica — como autômatos finitos e expressões regulares — e a estrutura da tabela de símbolos, essencial para as fases seguintes da compilação.

1.1 – Compilação x Interpretação

Diferenças, vantagens e desvantagens de cada abordagem

Linguagens de programação precisam ser traduzidas para código de máquina executável. Existem duas abordagens principais:

■ COMPILAÇÃO

Traduz todo o código-fonte de uma vez, gerando um arquivo executável (binário).

- ✓ Execução muito mais rápida
- ✓ Código distribuído sem fonte
- ✓ Otimizações profundas possíveis
- ✗ Compilação demorada para grandes projetos
- ✗ Executável é dependente de plataforma

Exemplos: C, C++, Go, Rust, Fortran

■ INTERPRETAÇÃO

Traduz e executa o código linha a linha, sem gerar arquivo intermediário.

- ✓ Portabilidade (roda em qualquer plataforma)
- ✓ Desenvolvimento e teste mais ágeis
- ✓ Tipagem dinâmica facilitada
- ✗ Execução mais lenta
- ✗ Código-fonte necessário para execução

Exemplos: Python, Ruby, JavaScript, PHP

■ PIPELINE: COMPILADOR VS INTERPRETADOR

COMPILADOR:



INTERPRETADOR:



■ ABORDAGEM HÍBRIDA (JIT):

Java e C# compilam para bytecode intermediário e usam JIT (Just-In-Time) para traduzir em tempo de execução — combinando vantagens de ambas.

O QUE É ANÁLISE LÉXICA?

A Análise Léxica (ou varredura) é a primeira fase de um compilador. O analisador léxico (lexer/scanner) lê o código-fonte como uma sequência de caracteres e o agrupa em unidades significativas chamadas TOKENS. Ele elimina espaços em branco, comentários e caracteres irrelevantes, entregando à próxima fase apenas tokens.

O QUE É UM TOKEN?

Um token é um par <tipo, valor> que representa a menor unidade léxica com significado:

IDENTIFICADOR	Nomes de variáveis, funções	x, total, calcular
PALAVRA-CHAVE	Reservadas pela linguagem	if, while, return
NÚMERO	Literais numéricos	42, 3.14, 0xFF
OPERADOR	Operadores aritm./lógicos	+, -, *, /, ==
DELIMITADOR	Pontuação estrutural	;, (,), {, }
STRING	Cadeias de caracteres	"hello", "abc"

EXEMPLO DE TOKENIZAÇÃO

int soma = a + b * 2;

Código-fonte de entrada:

int	soma	=	a	+	b	*	2	;
PALAVRA-CHAVE	IDENTIFICADOR	OPERADOR	IDENTIFICADOR	OPERADOR	IDENTIFICADOR	OPERADOR	NÚMERO	DELIMITADOR

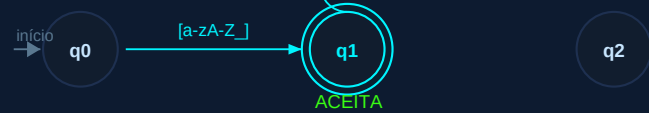
PROCESSO DO ANALISADOR LÉXICO

1. Leitura	Lê o código-fonte caractere por caractere (buffer de entrada)
2. Reconhecimento	Aplica regras (AFD / regex) para identificar padrões de tokens
3. Classificação	Determina o tipo do token reconhecido (palavra-chave, id, número, etc.)
4. Filtro	Elimina espaços, tabulações, quebras de linha e comentários
5. Emissão	Envia o par <tipo, lexema> para o analisador sintático

■ AUTÔMATOS FINITOS DETERMINÍSTICOS (AFD)

O AFD é o modelo matemático central na análise léxica. Cada categoria de token corresponde a um AFD que reconhece strings do padrão desejado. O lexer combina todos esses AFDs em um único autômato maior, percorrendo-o conforme lê caracteres.

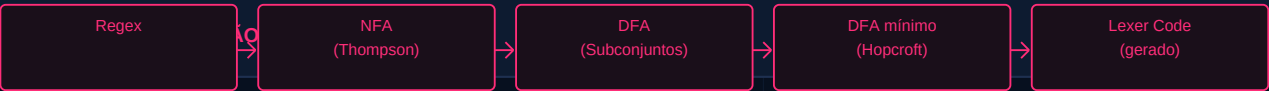
Formalmente: $M = (Q, \Sigma, \delta, q_0, F)$ onde:
 Q = conjunto finito de estados Σ = alfabeto de entrada
 δ = função de transição q_0 = estado inicial
 F = conjunto de estados de aceitação (cada um representa um tipo de token)



■ EXPRESSÕES REGULARES (REGEX)

Expressões regulares são a notação de alto nível para descrever padrões de tokens. Todo token pode ser descrito por uma regex, e toda regex pode ser convertida em AFD:

Identificador:	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>
Inteiro:	<code>[0-9]+</code>
Float:	<code>[0-9]+\.[0-9]+</code>
Operadores:	<code>[+\-*/=<>!=?]</code>
Comentário //	<code>\[/\[^\n]*</code>
String:	<code>"[^\"]*"</code>
Espaço (skip):	<code>[\t\n\r]+</code>



■ FERRAMENTAS: LEX, FLEX, ANTLR

- Lex/Flex:** Gerador de analisadores léxicos — recebe regras em regex e gera código C/C++.
- ANTLR4:** Gera lexers e parsers para múltiplas linguagens (Java, Python, C#, JS...).
- PLY:** Implementação de lex+yacc em Python puro — usada neste Case.

O QUE É A TABELA DE SÍMBOLOS?

A Tabela de Símbolos é uma estrutura de dados utilizada por todas as fases do compilador. Ela armazena informações sobre cada identificador encontrado no programa: variáveis, funções, constantes e tipos. É construída incrementalmente durante a análise léxica e enriquecida nas fases seguintes.

ESTRUTURA DA TABELA DE SÍMBOLOS

Campo	Descrição	Exemplo
LEXEMA	O nome/texto do identificador como aparece no código	a, calcular, PI
TIPO	Tipo do dado (int, float, string, função...)	int, float, void
ESCOPO	Nível de visibilidade (global, local, parâmetro)	global, local#1
LINHA	Linha do código-fonte onde foi declarado/usado	12, 34, 67
ENDEREÇO	Posição de memória alocada (fase de geração de código)	B, offset+4
ATRIBUTOS	Informações extras: nr. parâmetros, dimensões de arrays	ms=2, dim=[3,4]

EXEMPLO: TABELA GERADA PARA O CÓDIGO

```
int soma(int a, int b) { int resultado = a + b; return resultado; }
```

#	Lexema	Tipo	Escopo	Linha	Categoria
1	soma	função	global	1	FUNÇÃO
2	a	int	local	1	PARÂMETRO
3	b	int	local	1	PARÂMETRO
4	resultado	int	local	1	VARIÁVEL

IMPLEMENTAÇÃO E ESCOPO

- Implementação: Tipicamente uma hash table (O(1) médio) ou árvore AVL/B para múltiplos escopos.
- Escopo: Cada bloco { } cria um novo nível. Ao sair do bloco, entradas locais são removidas.
- Pilha de escopos: estrutura em pilha onde cada frame armazena a tabela do escopo atual.